Estimating Forest Size-Abundance Parameters

Adam Eichenwald

2026-03-03

Introduction

Understanding how trees are distributed by size within forests is fundamental to ecology, management, and conservation. Tree size distributions shape forest structure, carbon storage, biodiversity, and resilience. Across many forest types, smaller trees are far more numerous than larger trees, and mid-sized trees often follow a predictable **power-law (Pareto) distribution**, where abundance declines systematically with size.

Remote sensing offers a powerful way to capture forest structure over broad areas. **LiDAR** provides detailed vertical information but is often limited in coverage, while high-resolution **RGB imagery** is more widely available but primarily captures canopy trees. Because many mid-sized and understory trees may be occluded, simply counting visible trees can underestimate true forest structure.

The **ScalingFromSkypackage R package** leverages the known form of the size-abundance distribution to infer the obscured portion of the forest. By fitting a Pareto distribution to the observable portion of the data, the package interpolates the expected parameters for the **10–50 cm DBH range**, allowing estimation of both the **Pareto exponent (α)** and the **total number of trees (N_tot)**, including those potentially beneath the canopy and invisible in a remotely sensed image.

The workflow requires pre-processed tree-level data (DBH) and optionally site-specific covariates such as **leaf area index (LAI)**. It focuses on trees in the 10–50 cm DBH range, where the Pareto distribution generally holds, while smaller (<10 cm) and larger (>50 cm) trees naturally deviate from this pattern for ecological reasons, independent of remote sensing visibility (Eichenwald et al., 2025).

The workflow includes:

1. Identifying the observable range of tree sizes that follow a power-law distribution.
2. Estimating the Pareto exponent (α) for the 10–50 cm DBH range.
3. Estimating the total number of trees (N_tot) for the area of interest, including those potentially occluded.

This vignette demonstrates a complete workflow using sample data from the Harvard Forest ForestGEO project and NEON canopy height models. Although in our example the dataset was derived from canopy height rasters and segmentation of tree crowns, the methods in this package work with vector data (tree-level attributes such as DBH and height). The package itself does not process raw rasters; instead, it assumes a pre-processed dataset of individual crowns or trees.

Load Packages and Data

We begin by loading the necessary libraries, including `ScalingFromSkypackage`, and our sample dataset.

```
# library(sf)
library(dplyr)
```

```r
library(ggplot2) # Added for plotting
library(purrr)
library(lwgeom)
library(VGAM)
library(rstan)
library(posterior)
library(ScalingFromSkypackage)

# Load the sample data included with the package
data("harv_data_sample")
```

Step 1: Prepare Tree Data

ScalingFromSkypackage does not perform crown segmentation or DBH estimation itself; instead, these data are typically generated in earlier steps using other tools. For example, crown polygons can be segmented from LiDAR point clouds or high-resolution RGB imagery using packages such as lidR in R (Roussel et al., 2020) or DeepForest in Python (Weinstein et al., 2020), respectively. Geospatial packages such as sf in R can then import and analyze crown shapefiles, while allometric functions in itcSegment can be used to convert crown dimensions (e.g., height, area, perimeter) into DBH. In addition, site-level Leaf Area Index (LAI) is required to adjust estimates for potential segmentation errors. LAI can be obtained from field measurements or from remote-sensing products such as MODIS LAI or GEDI Plant Area Index (PAI).

Users can obtain crown segmentation data from Weinstein et al. (2024), available via Zenodo at https://doi.org/10.5281/zenodo.10926344, or from Weinstein et al. (2022) at https://doi.org/10.5281/zenodo.3459802. These datasets include delineated tree crowns and associated tree height estimates, making them suitable for calculating allometrically derived DBH values. The workflow for generating these crown shapefiles is well documented in Weinstein et al. (2024) and is based on methods originally developed in Weinstein et al. (2020). Together, these resources provide a reproducible foundation for linking remote sensing crown data to tree structural attributes such as DBH.

Users can also practice on the sample shapefile harvardshapefile included in this package. The shapefile represents a larger section of the Harvard Forest ForestGEO site from Weinstein et al. (2022) and includes UTM-referenced crown boundaries that can be used for demonstration purposes.

```r
# Load required packages
library(itcSegment)
```

```r
# Suppose `segmentedtrees` is your shapefile of tree crowns
# Make sure it has columns for height and geometry
# Convert geometry to numeric metrics
harvardshapefile$perimeter <- as.numeric(sf::st_perimeter(harvardshapefile))
harvardshapefile$area <- as.numeric(st_area(harvardshapefile))

# Estimate crown diameter from perimeter and area
harv_data_sample <- harvardshapefile %>%
  mutate(
    Diameter = 0.5 * sqrt(perimeter^2 - (8 * area)),
    # Calculate DBH using formula from Jucker et al. (2017)
```

```
    # Be sure to set `biome` to your forest type (default is global, which is indicated by the
         integer 0, but the equation can change depending on your forest location. See dbh help file
         for integer numbers and which biome they indicate)
    dbh = dbh(H = Max_Height, CA = Diameter, biome = 0)
  )%>%
  data.frame()

# `dbh` now contains estimated diameters at breast height for each tree
```

Once we have estimates of DBH for each segmented tree, we can begin the process of working with ScalingFromSkypackage. Here we load our pre-processed tree data, harv_data_sample, which contains attributes such as IDhectbest (a unique identifier for each hectare plot) and dbh (diameter at breast height). These DBH values were derived from crown segmentation and allometric conversion workflows similar to those described in Weinstein et al. (2020, 2021).

```
# Filter for a specific plot for a single-site example and extract the distribution of individual
         tree diameters
df_tile <- harv_data_sample %>%
  filter(IDhectbest == 1, !is.na(dbh)) %>%
  select(dbh)
```

Step 2: Determine Observation Window

The size-abundance distribution of trees often follows a power-law, but for data from remote sensing part of this distribution is hidden due to smaller trees being overshadowed by larger trees in the canopy. We use a kernel density estimation (KDE) and bootstrapping approach to identify the potential breakpoint at which the observable data deviates from the predicted distribution; trees smaller than this breakpoint are estimated to be obscured by the canopy. The `potential_break` function helps us to determine the point (x_{min}) where the observation window (where trees are expected to not be obscured) begins.

The function returns a named list with the following components:

- `potential_breakpoint`: **numeric scalar** giving the estimated lower log10 size bound for consistent detection.
- `bootstrap_kde_log`: **data frame** containing the log-scaled KDE curve (`log_x`, `mean_log_density`) used to identify peaks.
- `original_data_trimmed`: **numeric vector** of the DBH values after trimming by `trim_max`.
- `original_raw_data_df`: **data frame** containing the original, unmodified input data.
- `trim_max_value`: **numeric scalar** recording the `trim_max` value used in the function.

Note that df_tile is size data for just a single hectare plot within the Harvard Forest data. The size data df_tile is expected to be given in the form of a vector.

```
kde_output <- potential_break(df_tile)
```

We can inspect the output to see the estimated breakpoint.

```
# The result of this function is a list containing the breakpoint and KDE data.
# We'll print a summary of the key output.
cat("### Estimated Breakpoint\n")
```

Estimated Breakpoint

```
# The breakpoint is estimated on the log10 scale, so we exponentiate (10^x)
# to convert back to the original DBH units (cm).
cat(10^kde_output$potential_breakpoint, "cm")
```

26.36025 cm

This is only a first pass at the breakpoint, using a method that identifies peaks in the KDE. To verify and refine this estimate, we apply a second method that uses segmented regression on the bootstrapped KDE curve. Based on the initial potential breakpoint, the `determine_truncation_and_filter()` function refines the lower breakpoint (here, approximately 26 cm DBH) and identifies the upper truncation point (generally 50 cm DBH, as trees larger than this tend to deviate from the Pareto distribution).

This truncation defines the observational window for downstream analysis: only trees with DBH between the lower and upper bounds are used for fitting the power-law model. This ensures that the Bayesian model operates on the portion of the data that follows the expected distribution, avoiding extremes that are poorly predicted (e.g., DBH < 10 cm or >50 cm). Note that if a different distribution is used, truncating the upper bound may not be necessary.

The function takes as input the list returned by `potential_break()`, including the candidate breakpoint, the KDE curve, and the original dataset. The output is also a list with the following components:

- `breakpoint`: **numeric scalar** giving the finalized lower log10 size bound.
- `upper_bound`: **numeric scalar** giving the upper log10 size bound.
- `filtered_data`: **data frame** of the original tree data restricted to the observational window (10^breakpoint ≤ DBH ≤ 10^upper_bound).
- `bootstrap_kde_log`: **data frame** of the full log-scaled KDE from the first function, useful for plotting or inspection.

```
trunc_output <- truncate_filter(kde_output)
```

After running this function, trunc_output$bayesian_data contains the subset of trees within the consistent detection window and is used for fitting the truncated Pareto model in the next step.

Step 3: Parameter Estimation

Next, we fit a Bayesian Pareto model to estimate the power-law exponent, α, using the function `fit_alpha_model()`. This step accounts for site-level leaf area index (LAI), which influences detectability of smaller trees in remote sensing data, and allows incorporation of prior information about α, including uncertainty.

The main arguments of `fit_alpha_model()` are:

- `bayesian_data`: A **data frame** of tree sizes filtered to the observational window from `determine_truncation_and_filter()`. This ensures the model is fit only to sizes expected to follow a power-law distribution.
- `breakpoint`: The **numeric lower log10 size bound** for consistent detection, as determined by the previous truncation step.
- `LAI`: A **numeric value** for site-level leaf area index. In our study, we extracted LAI from MODIS data during leaf-out. The function expects LAI on a 1–10 scale; it normalizes this internally to 0–1 (e.g., enter

5 for a LAI of 5 on a 1–10 scale, or 50 on a 1–100 scale). Users outside the US can retrieve LAI from global MODIS products for their site.

- `prior_mean` and `prior_sd`: **Numeric values** specifying the mean and standard deviation of the prior for α. In our original workflow, we estimated this prior using a random forest model trained on FIA data, which predicts α given environmental covariates. For other regions, users can supply priors based on local inventory data or ecological knowledge.
- `stan_model_code`: A Stan model object or character string specifying the model to run (default = `stan_alpha_model`).
- `iter`, `warmup`, `chains`, `cores`, `refresh`: Standard Stan MCMC settings, specifying the number of iterations, warmup steps, number of chains, CPU cores, and output refresh frequency.

The function returns a **list** with:

- `posterior_summary`: A **data frame** summarizing the posterior distribution of α, including R^2 for model fit.
- `stan_fit`: The **full Stan fit object**, which can be inspected or used for further posterior analyses.

```
fit <- fit_alpha_model(
  filtered_data = trunc_output,
  LAI = 5.426,            # Example LAI value for the site
  prior_mean = 1.4,
  prior_sd = 0.3
)
#> Running "C:/PROGRA~1/R/R-45~1.0/bin/x64/Rcmd.exe" SHLIB foo.c
#> using C compiler: 'gcc.exe (GCC) 14.2.0'
#> gcc  -I"C:/PROGRA~1/R/R-45~1.0/include" -DNDEBUG   -I"C:/Users/adam.local/AppData/Local/R/win-
         library/4.5/Rcpp/include/"  -I"C:/Users/adam.local/AppData/Local/R/win-
         library/4.5/RcppEigen/include/"  -I"C:/Users/adam.local/AppData/Local/R/win-
         library/4.5/RcppEigen/include/unsupported"  -I"C:/Users/adam.local/AppData/Local/R/win-
         library/4.5/BH/include" -I"C:/Users/adam.local/AppData/Local/R/win-
         library/4.5/StanHeaders/include/src/"  -I"C:/Users/adam.local/AppData/Local/R/win-
         library/4.5/StanHeaders/include/"  -I"C:/Users/adam.local/AppData/Local/R/win-
         library/4.5/RcppParallel/include/" -DRCPP_PARALLEL_USE_TBB=1 -
         I"C:/Users/adam.local/AppData/Local/R/win-library/4.5/rstan/include" -DEIGEN_NO_DEBUG  -
         DBOOST_DISABLE_ASSERTS  -DBOOST_PENDING_INTEGER_LOG2_HPP  -DSTAN_THREADS  -DUSE_STANC3 -
         DSTRICT_R_HEADERS  -DBOOST_PHOENIX_NO_VARIADIC_EXPRESSION  -D_HAS_AUTO_PTR_ETC=0  -include
         "C:/Users/adam.local/AppData/Local/R/win-
         library/4.5/StanHeaders/include/stan/math/prim/fun/Eigen.hpp"  -std=c++1y    -
         I"C:/rtools45/x86_64-w64-mingw32.static.posix/include"      -O2 -Wall -std=gnu2x  -
         mfpmath=sse -msse2 -mstackrealign   -c foo.c -o foo.o
#> cc1.exe: warning: command-line option '-std=c++14' is valid for C++/ObjC++ but not for C
#> In file included from C:/Users/adam.local/AppData/Local/R/win-
         library/4.5/RcppEigen/include/Eigen/Core:19,
#>                  from C:/Users/adam.local/AppData/Local/R/win-
         library/4.5/RcppEigen/include/Eigen/Dense:1,
#>                  from C:/Users/adam.local/AppData/Local/R/win-
         library/4.5/StanHeaders/include/stan/math/prim/fun/Eigen.hpp:22,
#>                  from <command-line>:
#> C:/Users/adam.local/AppData/Local/R/win-
         library/4.5/RcppEigen/include/Eigen/src/Core/util/Macros.h:679:10: fatal error: cmath: No
         such file or directory
#>   679 | #include <cmath>
#>       |          ^~~~~~~
#> compilation terminated.
#> make: *** [C:/PROGRA~1/R/R-45~1.0/etc/x64/Makeconf:289: foo.o] Error 1
#>
#> SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
#> Chain 1:
```

```
#> Chain 1: Gradient evaluation took 7.4e-05 seconds
#> Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 0.74 seconds.
#> Chain 1: Adjust your expectations accordingly!
#> Chain 1:
#> Chain 1:
#> Chain 1: Iteration:    1 / 9000 [  0%]  (Warmup)
#> Chain 1: Iteration:  900 / 9000 [ 10%]  (Warmup)
#> Chain 1: Iteration: 1800 / 9000 [ 20%]  (Warmup)
#> Chain 1: Iteration: 2700 / 9000 [ 30%]  (Warmup)
#> Chain 1: Iteration: 3600 / 9000 [ 40%]  (Warmup)
#> Chain 1: Iteration: 4500 / 9000 [ 50%]  (Warmup)
#> Chain 1: Iteration: 5400 / 9000 [ 60%]  (Warmup)
#> Chain 1: Iteration: 6001 / 9000 [ 66%]  (Sampling)
#> Chain 1: Iteration: 6900 / 9000 [ 76%]  (Sampling)
#> Chain 1: Iteration: 7800 / 9000 [ 86%]  (Sampling)
#> Chain 1: Iteration: 8700 / 9000 [ 96%]  (Sampling)
#> Chain 1: Iteration: 9000 / 9000 [100%]  (Sampling)
#> Chain 1:
#> Chain 1:  Elapsed Time: 0.552 seconds (Warm-up)
#> Chain 1:                0.285 seconds (Sampling)
#> Chain 1:                0.837 seconds (Total)
#> Chain 1:
#>
#> SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 2).
#> Chain 2:
#> Chain 2: Gradient evaluation took 7.2e-05 seconds
#> Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.72 seconds.
#> Chain 2: Adjust your expectations accordingly!
#> Chain 2:
#> Chain 2:
#> Chain 2: Iteration:    1 / 9000 [  0%]  (Warmup)
#> Chain 2: Iteration:  900 / 9000 [ 10%]  (Warmup)
#> Chain 2: Iteration: 1800 / 9000 [ 20%]  (Warmup)
#> Chain 2: Iteration: 2700 / 9000 [ 30%]  (Warmup)
#> Chain 2: Iteration: 3600 / 9000 [ 40%]  (Warmup)
#> Chain 2: Iteration: 4500 / 9000 [ 50%]  (Warmup)
#> Chain 2: Iteration: 5400 / 9000 [ 60%]  (Warmup)
#> Chain 2: Iteration: 6001 / 9000 [ 66%]  (Sampling)
#> Chain 2: Iteration: 6900 / 9000 [ 76%]  (Sampling)
#> Chain 2: Iteration: 7800 / 9000 [ 86%]  (Sampling)
#> Chain 2: Iteration: 8700 / 9000 [ 96%]  (Sampling)
#> Chain 2: Iteration: 9000 / 9000 [100%]  (Sampling)
#> Chain 2:
#> Chain 2:  Elapsed Time: 0.553 seconds (Warm-up)
#> Chain 2:                0.305 seconds (Sampling)
#> Chain 2:                0.858 seconds (Total)
#> Chain 2:
#>
#> SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 3).
#> Chain 3:
#> Chain 3: Gradient evaluation took 2.8e-05 seconds
```

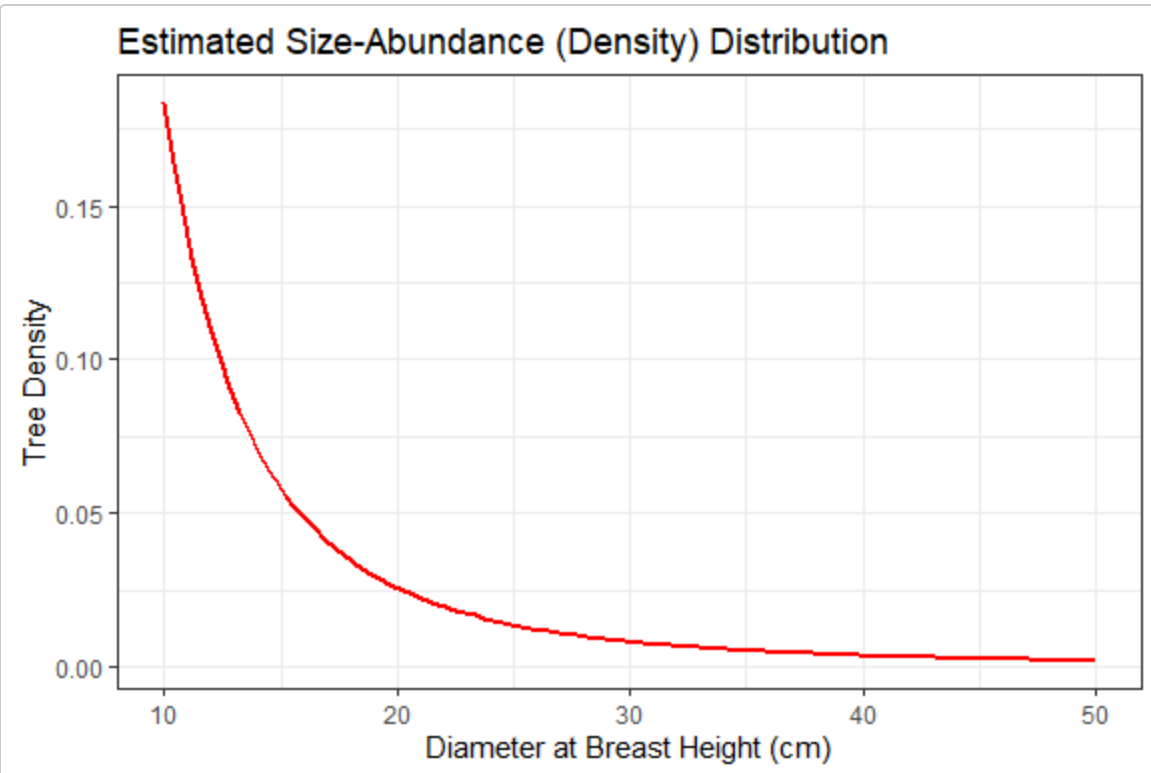
```
#> Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 0.28 seconds.
#> Chain 3: Adjust your expectations accordingly!
#> Chain 3:
#> Chain 3:
#> Chain 3: Iteration:    1 / 9000 [  0%]  (Warmup)
#> Chain 3: Iteration:  900 / 9000 [ 10%]  (Warmup)
#> Chain 3: Iteration: 1800 / 9000 [ 20%]  (Warmup)
#> Chain 3: Iteration: 2700 / 9000 [ 30%]  (Warmup)
#> Chain 3: Iteration: 3600 / 9000 [ 40%]  (Warmup)
#> Chain 3: Iteration: 4500 / 9000 [ 50%]  (Warmup)
#> Chain 3: Iteration: 5400 / 9000 [ 60%]  (Warmup)
#> Chain 3: Iteration: 6001 / 9000 [ 66%]  (Sampling)
#> Chain 3: Iteration: 6900 / 9000 [ 76%]  (Sampling)
#> Chain 3: Iteration: 7800 / 9000 [ 86%]  (Sampling)
#> Chain 3: Iteration: 8700 / 9000 [ 96%]  (Sampling)
#> Chain 3: Iteration: 9000 / 9000 [100%]  (Sampling)
#> Chain 3:
#> Chain 3:  Elapsed Time: 0.543 seconds (Warm-up)
#> Chain 3:                0.285 seconds (Sampling)
#> Chain 3:                0.828 seconds (Total)
#> Chain 3:
#>
#> SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 4).
#> Chain 4:
#> Chain 4: Gradient evaluation took 3.3e-05 seconds
#> Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 0.33 seconds.
#> Chain 4: Adjust your expectations accordingly!
#> Chain 4:
#> Chain 4:
#> Chain 4: Iteration:    1 / 9000 [  0%]  (Warmup)
#> Chain 4: Iteration:  900 / 9000 [ 10%]  (Warmup)
#> Chain 4: Iteration: 1800 / 9000 [ 20%]  (Warmup)
#> Chain 4: Iteration: 2700 / 9000 [ 30%]  (Warmup)
#> Chain 4: Iteration: 3600 / 9000 [ 40%]  (Warmup)
#> Chain 4: Iteration: 4500 / 9000 [ 50%]  (Warmup)
#> Chain 4: Iteration: 5400 / 9000 [ 60%]  (Warmup)
#> Chain 4: Iteration: 6001 / 9000 [ 66%]  (Sampling)
#> Chain 4: Iteration: 6900 / 9000 [ 76%]  (Sampling)
#> Chain 4: Iteration: 7800 / 9000 [ 86%]  (Sampling)
#> Chain 4: Iteration: 8700 / 9000 [ 96%]  (Sampling)
#> Chain 4: Iteration: 9000 / 9000 [100%]  (Sampling)
#> Chain 4:
#> Chain 4:  Elapsed Time: 0.592 seconds (Warm-up)
#> Chain 4:                0.273 seconds (Sampling)
#> Chain 4:                0.865 seconds (Total)
#> Chain 4:
```

Let’s create a plot showing the resulting fitted curve:

```
# Create a data frame for plotting the fitted line
plot_data <- tibble(
```

```
  dbh = seq(10, 50, length.out = 100),
  fit_n_dbh = dpareto(dbh, shape = fit$posterior_summary$mean, scale = 10)
)

# Plot the distribution and the fitted model
ggplot(trunc_output$bayesian_data, aes(x = dbh)) +
  geom_line(data = plot_data, aes(x = dbh, y = fit_n_dbh), color = "red", size = 1) +
  # scale_x_continuous(trans = 'log10') +
  # scale_y_continuous(trans = 'log10') +
  labs(title = "Estimated Size-Abundance (Density) Distribution",
       x = "Diameter at Breast Height (cm)",
       y = "Tree Density") +
  theme_bw()
```



Using the posterior samples of α from our fitted model, we estimate the total number of trees for the hectare plot. This gives us a final, corrected estimate of tree abundance for the site. The priors for the total number of trees (N_tot_prior_mean, N_tot_prior_sd) are set by default to 1250 and 625, respectively. These values come from estimates using TreeMap (Riley et al. 2021), a tree-level imputation of FIA plot data across the conterminous US. These defaults provide a reasonable prior for a typical 1 ha plot, but users may wish to adjust them to better reflect site-specific expectations.

Note: This chunk may take longer to run than the code estimating α.

```
trees <- estimate_total_trees(
  alpha_model_output = fit,
  N_tot_prior_mean = 1250,
  N_tot_prior_sd = 625
)
#> WARNING: Rtools is required to build R packages, but is not currently installed.
#>
```

```
#> Please download and install the appropriate version of Rtools for 4.5.0 from
#> https://cran.r-project.org/bin/windows/Rtools/.
#> Trying to compile a simple C file
#> Running "C:/PROGRA~1/R/R-45~1.0/bin/x64/Rcmd.exe" SHLIB foo.c
#> using C compiler: 'gcc.exe (GCC) 14.2.0'
#> gcc  -I"C:/PROGRA~1/R/R-45~1.0/include" -DNDEBUG   -I"C:/Users/adam.local/AppData/Local/R/win-
         library/4.5/Rcpp/include/"  -I"C:/Users/adam.local/AppData/Local/R/win-
         library/4.5/RcppEigen/include/"  -I"C:/Users/adam.local/AppData/Local/R/win-
         library/4.5/RcppEigen/include/unsupported"  -I"C:/Users/adam.local/AppData/Local/R/win-
         library/4.5/BH/include" -I"C:/Users/adam.local/AppData/Local/R/win-
         library/4.5/StanHeaders/include/src/"  -I"C:/Users/adam.local/AppData/Local/R/win-
         library/4.5/StanHeaders/include/"  -I"C:/Users/adam.local/AppData/Local/R/win-
         library/4.5/RcppParallel/include/" -DRCPP_PARALLEL_USE_TBB=1 -
         I"C:/Users/adam.local/AppData/Local/R/win-library/4.5/rstan/include" -DEIGEN_NO_DEBUG  -
         DBOOST_DISABLE_ASSERTS  -DBOOST_PENDING_INTEGER_LOG2_HPP  -DSTAN_THREADS  -DUSE_STANC3 -
         DSTRICT_R_HEADERS  -DBOOST_PHOENIX_NO_VARIADIC_EXPRESSION  -D_HAS_AUTO_PTR_ETC=0  -include
         "C:/Users/adam.local/AppData/Local/R/win-
         library/4.5/StanHeaders/include/stan/math/prim/fun/Eigen.hpp"  -std=c++1y    -
         I"C:/rtools45/x86_64-w64-mingw32.static.posix/include"      -O2 -Wall -std=gnu2x  -
         mfpmath=sse -msse2 -mstackrealign   -c foo.c -o foo.o
#> cc1.exe: warning: command-line option '-std=c++14' is valid for C++/ObjC++ but not for C
#> In file included from C:/Users/adam.local/AppData/Local/R/win-
         library/4.5/RcppEigen/include/Eigen/Core:19,
#>                  from C:/Users/adam.local/AppData/Local/R/win-
         library/4.5/RcppEigen/include/Eigen/Dense:1,
#>                  from C:/Users/adam.local/AppData/Local/R/win-
         library/4.5/StanHeaders/include/stan/math/prim/fun/Eigen.hpp:22,
#>                  from <command-line>:
#> C:/Users/adam.local/AppData/Local/R/win-
         library/4.5/RcppEigen/include/Eigen/src/Core/util/Macros.h:679:10: fatal error: cmath: No
         such file or directory
#>   679 | #include <cmath>
#>       |          ^~~~~~~
#> compilation terminated.
#> make: *** [C:/PROGRA~1/R/R-45~1.0/etc/x64/Makeconf:289: foo.o] Error 1
#> WARNING: Rtools is required to build R packages, but is not currently installed.
#>
#> Please download and install the appropriate version of Rtools for 4.5.0 from
#> https://cran.r-project.org/bin/windows/Rtools/.
```

The prior is informed by the TreeMap dataset, which provides tree density estimates (trees per acre, TPA) for the U.S. using a random forest machine-learning approach (Riley et al. 2021). The mean is selected to reflect central tendencies in the TreeMap data, whereas the standard deviation accounts for the spread in tree density estimates across different forests. Alternatively, this prior can also be informed by a smaller, in situ observed validation dataset from a subset of a given site.

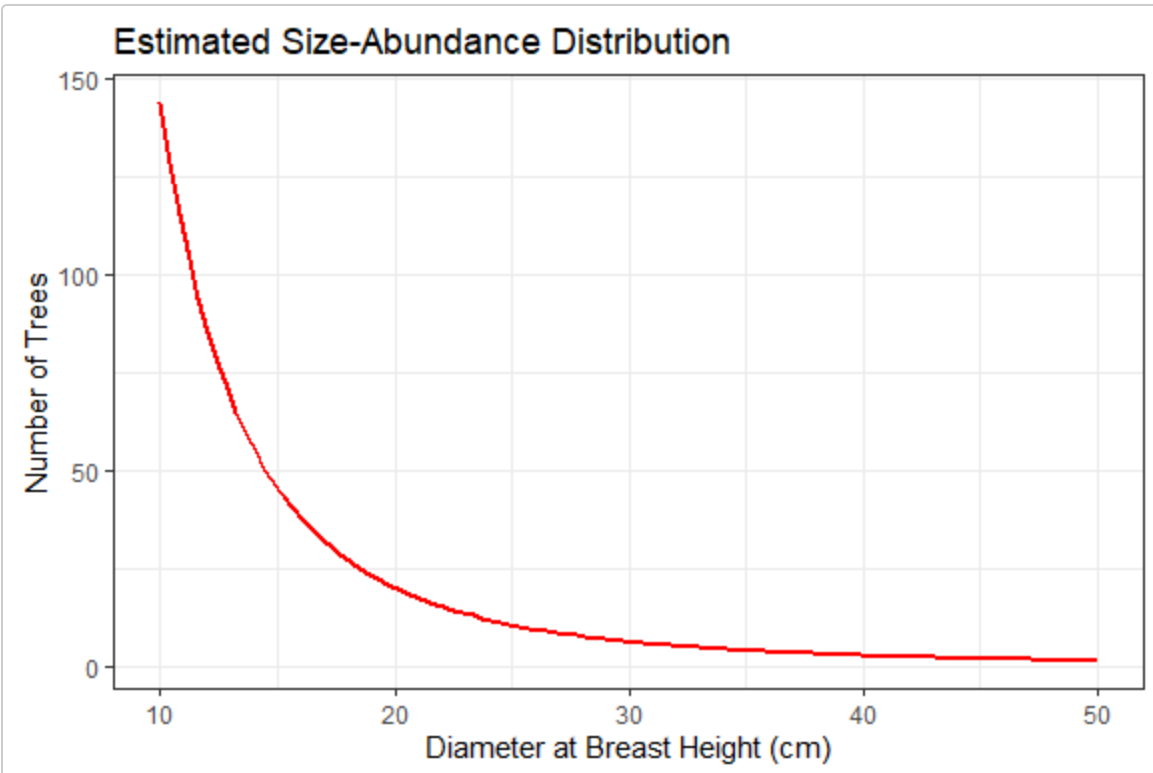
Step 4: Final Output

Visualize the results, where the y-axis density from our previous graph is converted to tree abundance.

```
# Create a data frame for plotting the fitted line
plot_data <- tibble(
  dbh = seq(10, 50, length.out = 100),
  fit_n_dbh = dpareto(dbh, shape = fit$posterior_summary$mean, scale = 10)*
         trees$posterior_summary$mean
```

```
)

# Plot the distribution and the fitted model
ggplot(trunc_output$bayesian_data, aes(x = dbh)) +
  geom_line(data = plot_data, aes(x = dbh, y = fit_n_dbh), color = "red", size = 1) +
  # scale_x_continuous(trans = 'log10') +
  # scale_y_continuous(trans = 'log10') +
  labs(title = "Estimated Size-Abundance Distribution",
       x = "Diameter at Breast Height (cm)",
       y = "Number of Trees") +
  theme_bw()
```



Extra Steps: Loop Over Sites

The entire workflow can be applied across multiple spatial units (tiles, plots, or stands) by simply iterating through them. In this case, the multiple plots are the 1 hectare subplots within the larger ForestGEO plot at Harvard Forest. The following loop demonstrates how you would apply the functions to multiple plots. The `tryCatch` blocks are used for robust error handling, ensuring the loop can continue even if one of the plots fails to meet the model's requirements. The first loop will likely take longer than any succeeding loop, since the Stan code has to compile the first time you run it in a session. Once it has compiled, however, the run should be a little faster.

```
# Initialize empty lists to store results
alpha_results <- list()  # will store posterior summaries for alpha (power-law exponent) for each
         plot
tree_results <- list()   # will store posterior summaries for total number of trees (N_tot) for each
         plot

# Loop over each unique hectare plot in the sample dataset
for (i in unique(harv_data_sample$IDhectbest)) {
```

```r
  # Subset the data for the current plot and remove rows with missing DBH
  df_tile <- harv_data_sample %>%
    filter(IDhectbest == i, !is.na(dbh))

  # Skip plots with too few trees for meaningful analysis
  if (nrow(df_tile) < 25) next

  df_tile<-df_tile%>%pull(dbh)

  # Step 1: Identify potential breakpoint using KDE and bootstrapping
  # Wrap in tryCatch to skip plots where the function fails
  kde_output <- tryCatch(
    potential_break(df_tile),
    error = function(e) NULL  # return NULL if an error occurs
  )
  if (is.null(kde_output)) next  # skip to next plot if KDE fails

  # Step 2: Refine breakpoint and determine upper truncation
  trunc_output <- tryCatch(
    truncate_filter(kde_output),
    error = function(e) NULL
  )
  if (is.null(trunc_output)) next  # skip if truncation fails

  # Step 3: Fit Bayesian Pareto model to estimate alpha
  fit <- tryCatch(
    fit_alpha_model(
      bayesian_data = trunc_output$bayesian_data,        # filtered data within observation window
      breakpoint = trunc_output$final_breakpoint,       # refined lower bound
      LAI = 3.5,                                        # leaf area index for the site
      prior_mean = 1.4,                                 # prior mean for alpha
      prior_sd = 0.3                                    # prior SD for alpha
    ),
    error = function(e) NULL
  )
  if (is.null(fit)) next  # skip if Bayesian model fails

  # Step 4: Estimate total number of trees (N_tot) using posterior samples of alpha
  trees <- tryCatch(
    estimate_total_trees(fit),
    error = function(e) NULL
  )
  if (is.null(trees)) next  # skip if total tree estimation fails

  # Store the results in the lists, using a unique plot identifier
  alpha_results[[paste0("HARV_", i)]] <- fit$posterior_summary  # posterior summary for alpha
  tree_results[[paste0("HARV_", i)]] <- trees$posterior_summary # posterior summary for N_tot
}
```

Extra Steps: Parallelization

If you have a lot of sites to run (like if you are developing a raster of size-abundance distribution for a large area and want to individually interpolate the distribution for each cell) then you should likely parallelize this code. Packages such as *foreach*, *doParallel*, or *parallel* could work for you here. There may also be a way to vectorize this approach, which might be faster than a *for loop* since it is more in line with how R is supposed to operate.

```r
# Load necessary packages for parallel processing
library(foreach)      # provides foreach loops for parallel execution
library(doParallel)   # allows registering parallel backends

# Detect the number of CPU cores available, and leave one core free for system processes
n_cores <- parallel::detectCores() - 1
cl <- makeCluster(n_cores)    # create a cluster object with the available cores
registerDoParallel(cl)         # register the cluster for use with foreach

# Initialize lists to store results after the loop
alpha_results <- list()  # posterior summaries for alpha (power-law exponent)
tree_results <- list()   # posterior summaries for total number of trees (N_tot)

# Parallelized foreach loop
# Each iteration runs in parallel on a different core
alpha_tree_results <- foreach(
  i = unique(harv_data_sample$IDhectbest),      # loop over unique hectare plot IDs
  .packages = c("dplyr", "ScalingFromSkypackage")  # packages needed in each worker
) %dopar% {   # %dopar% tells foreach to execute in parallel instead of sequentially

  # Subset the data for the current plot and remove rows with missing DBH
  df_tile <- harv_data_sample %>% filter(IDhectbest == i, !is.na(dbh))

  # Skip plots with too few trees
  if (nrow(df_tile) < 25) return(NULL)

  # Step 1: Identify potential breakpoint using KDE and bootstrapping
  kde_output <- tryCatch(
    potential_break(df_tile),
    error = function(e) NULL   # return NULL if the function fails
  )
  if (is.null(kde_output)) return(NULL)

  # Step 2: Refine breakpoint and determine upper truncation
  trunc_output <- tryCatch(
    truncate_filter(kde_output),
    error = function(e) NULL
  )
  if (is.null(trunc_output)) return(NULL)

  # Step 3: Fit Bayesian Pareto model to estimate alpha
  fit <- tryCatch(
    fit_alpha_model(
      trunc_output,
      LAI = 3.5,                        # leaf area index
```

```
      prior_mean = 1.4,                 # prior mean for alpha
      prior_sd = 0.3                     # prior SD for alpha
    ),
    error = function(e) NULL
  )
  if (is.null(fit)) return(NULL)

  # Step 4: Estimate total number of trees (N_tot) using posterior samples of alpha
  trees <- tryCatch(
    estimate_total_trees(fit),
    error = function(e) NULL
  )
  if (is.null(trees)) return(NULL)

  # Return both alpha and tree results as a list
  list(alpha = fit$posterior_summary, trees = trees$posterior_summary)
}

# Combine results from parallel execution into named lists
for (i in seq_along(alpha_tree_results)) {
  if (!is.null(alpha_tree_results[[i]])) {
    site_name <- paste0("HARV_", unique(harv_data_sample$IDhectbest)[i])
    alpha_results[[site_name]] <- alpha_tree_results[[i]]$alpha
    tree_results[[site_name]] <- alpha_tree_results[[i]]$trees
  }
}

# Stop the parallel cluster when finished to free system resources
stopCluster(cl)
```

Literature Cited

Eichenwald, A. J., Grady, J. M., Knott, J. A., Rodriguez, J. M., Weinstein, B. G., Orwig, D. A., & Record, S. (2025). Leveraging Remote Sensing and Theory to Predict Tree Size Abundance Distributions Across Space. Global Ecology and Biogeography, 34(8)

Riley, K. L., I. C. Grenfell, M. A. Finney, and J. M. Wiener. 2021. “TreeMap, a Tree-Level Model of Conterminous US Forests Circa 2014 Produced by Imputation of FIA Plot Data.” Scientific Data 8: 11.

Weinstein, B. G., Marconi, S., Aubry-Kientz, M., Vincent, G., Senyondo, H., & White, E. P. (2020). DeepForest: A Python package for RGB deep learning tree crown delineation. Methods in Ecology and Evolution, 11(12), 1743–1751. https://doi.org/10.1111/2041-210X.13472

Weinstein, B. G., Marconi, S., Bohlman, S. A., Zare, A., Singh, A., Graves, S. J., & White, E. P. (2021). A remote sensing derived data set of 100 million individual tree crowns for the National Ecological Observatory Network. eLife, 10, e62922. https://doi.org/10.7554/eLife.62922

Ben Weinstein, Sergio Marconi, & Ethan White. (2022). Data for the NeonTreeEvaluation Benchmark (0.2.2) [Data set]. Zenodo. https://doi.org/10.5281/zenodo.5914554

Weinstein, B. G., Marconi, S., Zare, A., Bohlman, S. A., Singh, A., Graves, S. J., Magee, L., Johnson, D. J., Record, S., Rubio, V. E., Swenson, N. G., Townsend, P. A., Veblen, T. T., Andrus, R. A., & White, E. P. (2024).

Individual tree species maps for the National Ecological Observatory Network. PLoS Biology, 22(7).
https://doi.org/10.1371/journal.pbio.3002700